4428 HW3

March 10, 2026

Ram Prasad

03-06-2025

1.

I extend the setup from the example. Let B be the number of blue whales, F the number of fin whales, g_B the blue whale population growth rate, g_F the fin whale growth rate, c_B the effect of competition on blue whales, and c_F the effect of competition on fin whales. I assume $B, F \geq 0$, and $\alpha \in \mathbb{R}_+$. The minimum viable populations are 3,000 and 15,000 for the blue and fin whale populations, respectively.

(a) I solve the system:

$$\frac{dB}{dt} = f_1(B,F) = g_B - c_B = 0.05B\left(\frac{B-3000}{B+3000}\right)\left(1-\frac{B}{150000}\right) - \alpha BF = 0$$
$$\frac{dF}{dt} = f_2(B,F) = g_F - c_F = 0.08F\left(\frac{F-15000}{F+15000}\right)\left(1-\frac{F}{400000}\right) - \alpha BF = 0$$

Checking the boundary cases of $B = 0$ and $F = 0$ immediately gives several equilbrium points:

$$(0,0), \quad (150000,0), \quad (0,400000), \quad (3000,0), \quad (0,15000).$$

The populations can coexist if there is an interior solution to this system, meaning an equilibrium point where it is simultaneously the case that $f_1(B,F) = 0$ and $f_2(B,F) = 0$ and $B > 0$ and $F > 0$ (neither population has gone extinct). It suffices to find at least one such point to prove the possibility of coexistence.

For the interior points, I assume $B > 0$ and $F > 0$. I divide each equation by B and F, respectively, and solve:

$$0.05\left(\frac{B-3000}{B+3000}\right)\left(1-\frac{B}{150000}\right) = \alpha F$$
$$0.08\left(\frac{F-15000}{F+15000}\right)\left(1-\frac{F}{400000}\right) = \alpha B$$

Each equation is cubic. Using a range from 0 up to the carrying capacity for blue and fin whales, I construct a grid of starting points for numpy's fsolve and identify 9 equlibrium points.

[1]:
```python
# standard imports
import numpy as np
import scipy as sp
import sympy as sym
import matplotlib.pyplot as plt
import pandas as pd

# suppress runtime warnings
import warnings
warnings.filterwarnings("ignore", category=RuntimeWarning)

# define population growth function
def G(r, P, c, K): return r*P*((P-c)/(P+c))*(1-P/K)

# define competition cost
def C(B, F, a): return a*B*F

# set up dynamical system
def whale_system(B_F, K_b=150_000, K_f=400_000,
                 C_b=3_000, C_f=15_000, a=10**-8, r_b=0.05, r_f=0.08):
  """
  Dynamical system for whale growth model.
  Args:
  -----
    B_F (tuple): blue and finned whale population
    K_b (float): blue carrying capacity
    K_f (float): fin carrying capacity
    C_b (float): blue minimum viable population
    C_f (float): fin minimum viable population
    a (float): positive real constant
    r_b (float): blue intrinsic growth rate
    r_f (float): fin intrinsic growth rate
  Returns:
  --------
    tuple: g(B)-c(B), g(F)-c(F)
  Function wraps population growth and competition
  cost functions.
  """
  # unpack population vector
  B, F = B_F

  # return growth - cost vector
  return [G(r_b, B, C_b, K_b) - C(B,F,a),
          G(r_f, F, C_f, K_f) - C(B,F,a)]


# grid for starting points: 0 to carrying capacity
```

```python
b0_grid = np.linspace(0,150000, 16)
f0_grid = np.linspace(0,400000, 41)

# track results
opt_results = []

# loop through starting points
for b0 in b0_grid:
  for f0 in f0_grid:

    # solve F(B,F) = 0 using given starting population
    BF_l = list(sp.optimize.fsolve(whale_system, x0=(b0,f0)))

    # check solution
    eq_l = whale_system(BF_l)

    # store result if valid (within tol of 0)
    if np.allclose(eq_l, 0, atol=1e-8):

      # enforce non-negativity
      if BF_l[0] >= -1e-8 and BF_l[1] >= -1e-8:

        # clip tiny negative zeros
        BF_l = [max(0,x) for x in BF_l]

        # update results
        opt_results.append([b0, f0, BF_l[0], BF_l[1]])

# results as dataframe
opt_results = pd.DataFrame(opt_results, columns=["B0","F0","B*","F*"])

# round to integer counts and drop duplicates
opt_results = opt_results.round({"B*":1, "F*":1})
opt_results = opt_results.drop_duplicates(subset=["B*","F*"])

# print eq points
print("=== Equilibrium Points ===")
display(opt_results)
```

```
=== Equilibrium Points ===
```

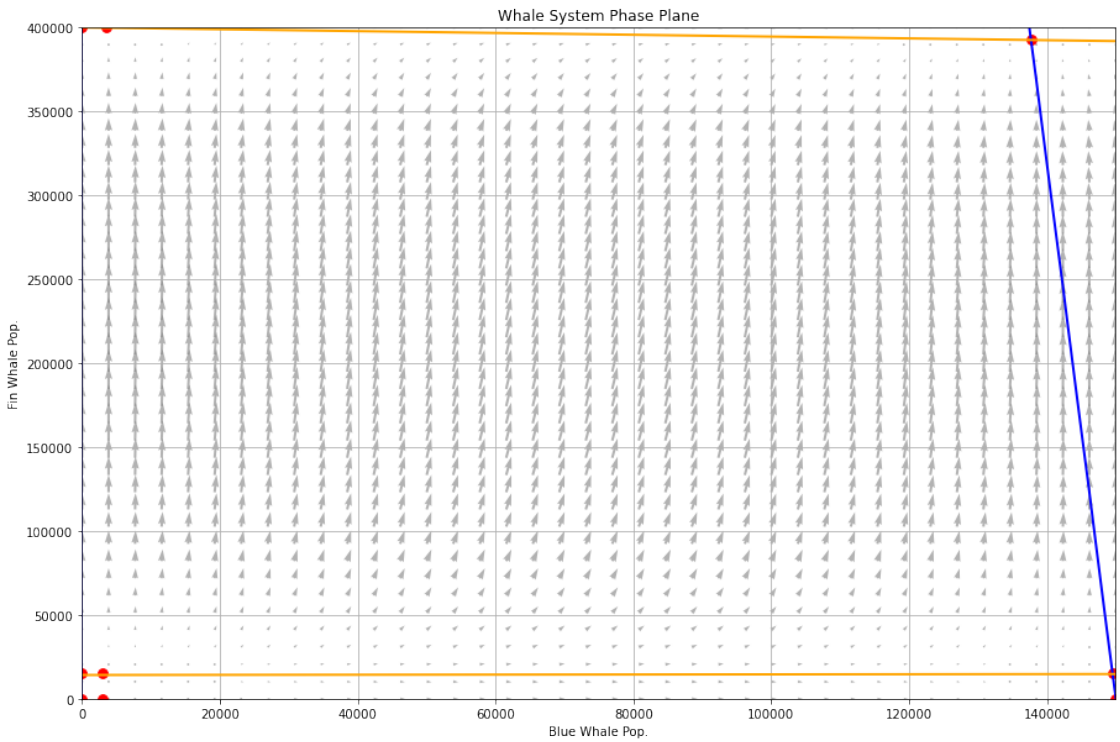
	B0	F0	B*	F*
0	0.0	0.0	0.0	0.0
1	0.0	10000.0	0.0	15000.0
21	0.0	220000.0	0.0	400000.0
38	10000.0	0.0	3000.0	0.0
39	10000.0	10000.0	3018.4	15011.8
54	10000.0	210000.0	3535.2	399809.5

```
105  30000.0  120000.0  149513.0   15595.0
114  30000.0  260000.0  137698.4  392568.0
221  80000.0       0.0  150000.0       0.0
```

Yes, the populations can coexist. There are 4 interior points, and subsequent stability analysis indicates at least 1 is stable.

(b) I sketch the vector field below, and include plots of equilibrium points and nullclines where each growth system is 0.

[2]:
```
# meshgrid for vector field
b_vals = np.linspace(0, 150000, 40)
f_vals = np.linspace(0, 400000, 40)
B, F = np.meshgrid(b_vals, f_vals)

# compute growth dynamics on grid
dB = G(0.05, B, 3_000, 150_000) - C(B, F, 10**-8)
dF = G(0.08, F, 15_000, 400_000) - C(B, F, 10**-8)

# plot vector field
plt.figure(figsize=(15,10))
plt.quiver(B, F, dB, dF, color="gray", alpha=0.6)

# plot nullclines where db=0, df=0
plt.contour(B, F, dB, levels=[0], colors="blue", linewidths=2)
plt.contour(B, F, dF, levels=[0], colors="orange", linewidths=2)

# scatter equilibrium points
for b, f in zip(opt_results["B*"], opt_results["F*"]):
  plt.scatter(b, f, marker="o", color="red", s=60)

# labels, title, grid
plt.xlabel("Blue Whale Pop.")
plt.ylabel("Fin Whale Pop.")
plt.title("Whale System Phase Plane")
plt.grid()
plt.show()
```



The phase plane above shows the vector field and nullclines. There are 9 intersection points, marked with red dots, and the field shows trajectories point towards the stable nodes.

For stability, I evaluate the Jacobian at each equilbrium point. Owing to the nonlinearity of the system, I use sympy's symbolic Jacobian to get the matrix analytically.

[3]:
```
# define symbols and functions
B, F = sym.symbols("B F")
f1 = 0.05*B*((B-3000)/(B+3000))*(1-B/150000) - 1e-8*B*F
f2 = 0.08*F*((F-15000)/(F+15000))*(1-F/400000) - 1e-8*B*F

# compute Jacobian
J = sym.Matrix([f1, f2]).jacobian([B,F])
print("=== Jacobian: ===")
display(J)

# loop through eq. points
stab_results, r1_results, r2_results = [], [], []
for b, f in zip(opt_results["B*"], opt_results["F*"]):

  # evaluate Jacobian at each eq. point
  j_eq = J.subs({B:b, F:f})

  # compute eigenvalues
```

```python
    e_val = np.linalg.eigvals(np.array(j_eq, dtype=float))

    # capture real parts of evalues
    e1, e2 = e_val[0], e_val[1]
    r1, r2 = np.real(e1), np.real(e2)

    """
    classify stability:
      stable: negative real parts
      unstable: positive real parts
      otherwise, saddle point
    """
    if r1 < 0 and r2 < 0:
      s = "Stable"
    elif r1 > 0 and r2 > 0:
      s = "Unstable"
    else:
      s = "Saddle"

    # update results
    stab_results.append(s)
    r1_results.append(r1)
    r2_results.append(r2)


# update DF and display
print("\n=== Stability Analysis: ===")
opt_results["R1"] = r1_results
opt_results["R2"] = r2_results
opt_results["Stability"] = stab_results
display(opt_results)
```

```
=== Jacobian: ===
```

$$\left[\begin{matrix} -\frac{0.05B\left(1-\frac{B}{150000}\right)(B-3000)}{(B+3000)^2} + \frac{0.05B\left(1-\frac{B}{150000}\right)}{B+3000} - \frac{3.33333333333333\cdot 10^{-7}B(B-3000)}{B+3000} - 1.0\cdot 10^{-8}F + \frac{0.05\cdot\left(1-\frac{B}{150000}\right)(B-3000)}{B+3000} \\ -1.0\cdot 10^{-8}F \end{matrix}\right. -$$

```
=== Stability Analysis: ===
```

	B0	F0	B*	F*	R1	R2	Stability
0	0.0	0.0	0.0	0.0	-0.050000	-0.080000	Stable
1	0.0	10000.0	0.0	15000.0	0.038500	-0.050150	Saddle
21	0.0	220000.0	0.0	400000.0	-0.074217	-0.054000	Stable
38	10000.0	0.0	3000.0	0.0	0.024500	-0.080030	Saddle
39	10000.0	10000.0	3018.4	15011.8	0.024493	0.038498	Unstable
54	10000.0	210000.0	3535.2	399809.5	0.024152	-0.074178	Saddle
105	30000.0	120000.0	149513.0	15595.0	-0.047873	0.038368	Saddle

```
114  30000.0  260000.0  137698.4  392568.0 -0.043585 -0.072815    Stable
221  80000.0       0.0  150000.0       0.0 -0.048039 -0.081500    Stable
```

The table above shows the classification of each equilibrium point using the Jacobian's eigenvalues. There is one stable interior equilibrium point, where B=137698.4 and F=392568.0.

(c) I use scipy's IVP solver to predict the behavior of the population with an initial guess of 5,000 and 70,000.

[4]:
```python
# wrapper function for IVP
def whale_ivp(t, y, **kwargs): return whale_system(y, **kwargs)

# solve IVP using 5000 and 70000 as starting point
ivp_sol = sp.integrate.solve_ivp(lambda t, y: whale_ivp(t,y, C_b=3000,␣
 ↪C_f=15000), [0,1000], [5_000,70_000])

# check results
print(ivp_sol.message)
if ivp_sol.success:

    # if successful, print solution
    print("\n=== IVP Sol'n: ===")
    print(ivp_sol.y[:,-1])

else:

    # if not, print last value
    print(f"=== Terminal Value at T={len(ivp_sol.t)}: ===")
    print(ivp_sol.y[:,-1])
```

```
The solver successfully reached the end of the integration interval.

=== IVP Sol'n: ===
[137696.30784767 392524.09333168]
```

The model predicts the $\boxed{\text{system eventually converges to the equilbirum at B=137,698 and F=392,568}}$ (the reported IVP is a bit short owing to the truncated integration interval), meaning both populations increase. Per the table above, this point is stable.

(d) I rerun the analysis, setting the minimum viable population for the blue whale to 10,000.

[5]:
```python
# solve IVP with blue mvp at 10000
ivp_sol_d = sp.integrate.solve_ivp(lambda t, y: whale_ivp(t, y, C_b=10_000,␣
 ↪C_f=15000), [0, 1000], [5_000, 70_000])

# check results
print(ivp_sol_d.message)

if ivp_sol_d.success:
```

```
    # print solution
    print("\n=== IVP Sol'n (C_b = 10,000): ===")
    print(ivp_sol_d.y[:,-1])

else:

    # print terminal value
    print(f"=== Terminal Value at T={len(ivp_sol_d.t)}: ===")
    print(ivp_sol_d.y[:,-1])
```

```
The solver successfully reached the end of the integration interval.

=== IVP Sol'n (C_b = 10,000): ===
[2.01481797e-07 3.99999683e+05]
```

The initial blue whale population is below the minimum viable population ($5000 < 10000$), so the $\boxed{\text{blue whale population goes extinct}}$, and the fin whale equation becomes a single species model in which the $\boxed{\text{fin population goes to the carrying capacity maximum of 400,000}}$.

2.

I solve the linear system

$$\begin{aligned}\dot{x} &= 4x - y \\ \dot{y} &= 2x + y\end{aligned}$$

subject to initial condition $(x_0, y_0) = (3, 4)$.

(a) The Jacobian is

$$A = \begin{pmatrix} 4 & -1 \\ 2 & 1 \end{pmatrix}.$$

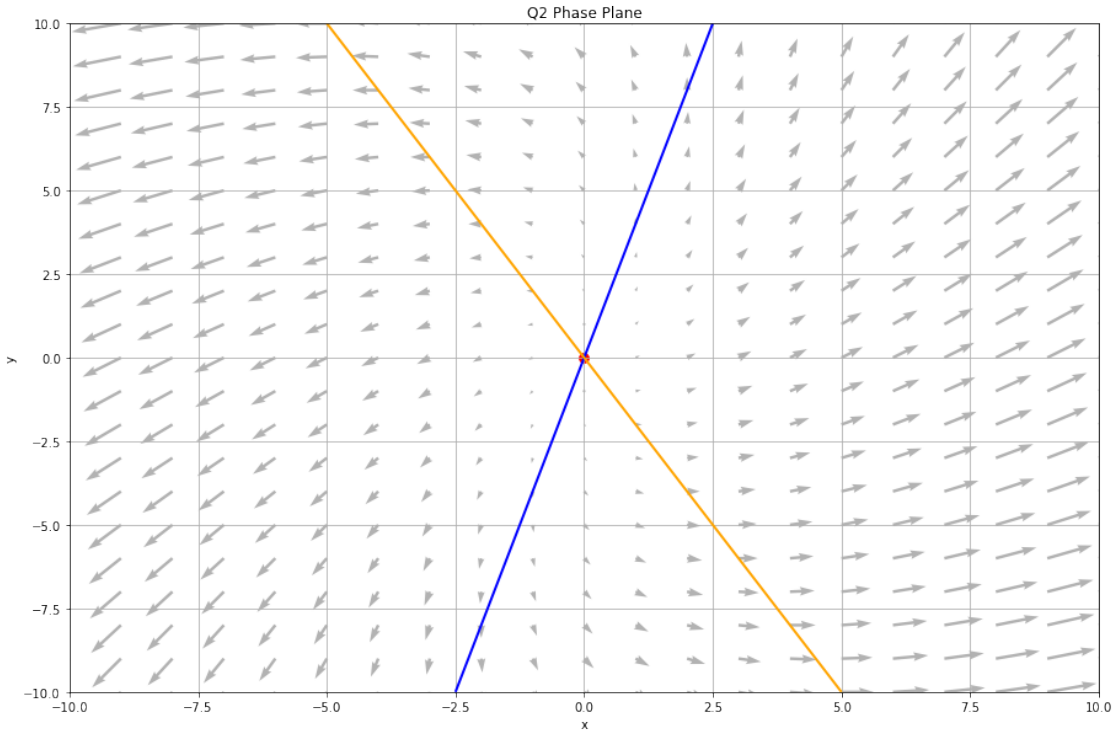
I solve the characteristic equation $\mid A - \lambda I \mid = 0$

$$\begin{aligned}\det \begin{pmatrix} 4-\lambda & -1 \\ 2 & 1-\lambda \end{pmatrix} &= 0 \\ (4-\lambda)(1-\lambda) + 2 &= 0 \\ 6 - 5\lambda + \lambda^2 &= 0 \\ (\lambda - 2)(\lambda - 3) = 0 &\implies \lambda = 2, 3.\end{aligned}$$

For $\boxed{\lambda = 2}$, the associated eigenvector is $\boxed{(1, 2)^\top}$, and for $\boxed{\lambda = 3}$, the associated eigenvector is $\boxed{(1, 1)^\top}$.

(b) The vector field is below. The Jacobian is constant, with 2 real, positive eigenvalues. Thus $\boxed{\text{the sole equilibrium (0,0) is an unstable node}}$.

[6]:
```
# meshgrid for vector field
x_vals = np.linspace(-10, 10, 21)
y_vals = np.linspace(-10, 10, 21)
X, Y = np.meshgrid(x_vals, y_vals)

# evaluate diff eq on grid
dX = 4*X - Y
dY = 2*X + Y

# plot vector field
plt.figure(figsize=(15,10))
plt.quiver(X, Y, dX, dY, color="gray", alpha=0.6)

# plot nullclines where db=0, df=0
plt.contour(X, Y, dX, levels=[0], colors="blue", linewidths=2)
plt.contour(X, Y, dY, levels=[0], colors="orange", linewidths=2)

# scatter equilibrium points
plt.scatter(0, 0, marker="o", color="red", s=60)

# labels, title, grid
plt.xlabel("x")
plt.ylabel("y")
plt.title("Q2 Phase Plane")
plt.grid()
plt.show()
```



(c) I solve the system analytically, subject to the initial condition $(x_0, y_0) = (3, 4)$.

Equilibrium occurs when $\dot{x} = 0$ and $\dot{y} = 0$ simultaneously. Using the eigenvalues and vectors computed above, the solution for the linear system takes the form

$$\begin{pmatrix} x \\ y \end{pmatrix} = c_1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} e^{2t} + c_2 \begin{pmatrix} 1 \\ 1 \end{pmatrix} e^{3t}.$$

I use the given initial condition to solve for c_1, c_2. Evaluating the system at $t = 0$ gives

$$\begin{pmatrix} 3 \\ 4 \end{pmatrix} = c_1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

thus

$$\begin{aligned} c_1 + c_2 &= 3 \\ 2c_1 + c_2 &= 4 \implies c_1 = 1, c_2 = 2. \end{aligned}$$

The solution to the system is

$$\boxed{x(t) = e^{2t} + 2e^{3t}, \quad y(t) = 2e^{2t} + 2e^{3t}}.$$

3.

The kinetics equation for x is

$$\dot{x} = k_1 a x - k_{-1} x^2,$$

where $k_1, k_{-1} > 0$, and a is treated as a constant.

(a) The kinetic model follows from the nature of the chemical reaction. The forward reaction $A+X \to 2X$ has a rate proportional to ax, hence the $+k_1 ax$ term. The sign is positive because the autocatalysis process creates a new molecule X. The backward reaction $2X \to A + X$ is proportional to x^2, hence the $-k_{-1}x^2$ term. The back reaction limits the growth in X, hence the negative sign.

(b) To identify fixed points, I set $\dot{x} = 0$. An immediate solution is $\boxed{x = 0}$. The other is

$$\begin{aligned} k_1 a x - k_{-1} x^2 &= 0 \\ x(k_1 a - k_{-1} x) &= 0 \\ k_1 a - k_{-1} x = 0 &\implies \boxed{x = \frac{k_1 a}{k_{-1}}}. \end{aligned}$$

For stability, I check the derivative

$$f'(x) = k_1 a - 2k_{-1} x$$

at each equilibrium point:

$$\begin{aligned} f'(0) &= k_1 a > 0 \implies \boxed{\text{Unstable.}} \\ f'\left(\frac{k_1 a}{k_{-1}}\right) &= k_1 a - 2k_{-1}\frac{k_1 a}{k_{-1}} \\ &= -k_1 a < 0 \implies \boxed{\text{Stable.}} \end{aligned}$$

(c) I use the general solution for a logistic ODE

$$\begin{aligned} \frac{dx}{dt} &= k_1 a x - k_{-1} x^2 \\ &= k_1 a x (1 - \frac{x}{k_1 a / k_{-1}}) \end{aligned}$$

which takes the form

$$x(t) = \frac{k_1 a / k_{-1}}{1 + C e^{-k_1 a t}}.$$

Let $x(0) = x_0$. Solving for c gives

$$
\begin{aligned}
x_0 = x(0) &= \frac{k_1 a/k_{-1}}{1+C} \\
1 + C &= \frac{k_1 a/k_{-1}}{x_0} \\
C &= \frac{k_1 a/k_{-1}}{x_0} - 1 \\
&= \frac{k_1 a/k_{-1} - x_0}{x_0}
\end{aligned}
$$

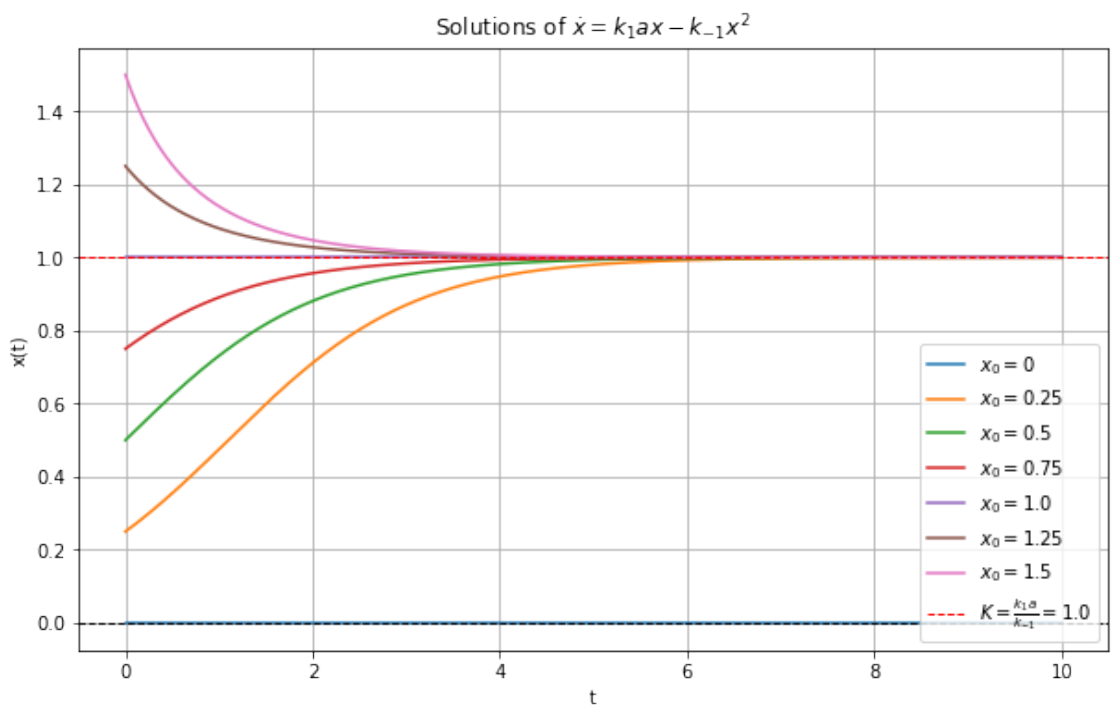
Thus

$$
x(t) = \frac{k_1 a/k_{-1}}{1 + \left(\frac{k_1 a/k_{-1} - x_0}{x_0}\right) e^{-k_1 a t}}.
$$

I plot $x(t)$, varying x_0, setting specific parameter values (the exact value isn't important to characterize behavior) for plotting.

[7]:
```
# set parameters
k1, a, k_1 = 1, 1, 1

# compute eq point
x_star = k1*a/k_1

# define logistic function
def x_t(t, x0, k1=1, a=1, k_1=1):
  """
  Logistic ODE.
  Args:
  -----
    t (float): time index
    x0 (float): initial value
    k1 (float): parameter
    a (float): paramter
    k_1 (float): parameter
  Returns:
  --------
    (float): x(t)
  See function above.
  """
  # return 0s if x0 is 0
  if x0 == 0:
    return np.zeros_like(t)

  # compute numerator
```

```
  K = k1*a/k_1

  # evaluate x(t)
  return K / (1 + ((K - x0)/x0)*np.exp(-k1*a*t))

# set time grid and initial values
t_grid = np.linspace(0, 10, 500)
x0_grid = [0, 0.25, 0.5, 0.75, 1.00, 1.25, 1.50]

# plot solutions
plt.figure(figsize=(10,6))
for x0 in x0_grid:
  plt.plot(t_grid, x_t(t_grid, x0, k1, a, k_1), label=fr"$x_0={x0}$")

# plot eq. lines
plt.axhline(0, color="black", linestyle="--", linewidth=1)
plt.axhline(x_star, color="red", linestyle="--", linewidth=1,␣
 ↪label=fr"$K=\frac{{k_1 a}}{{k_{{-1}}}}={x_star}$")

# labels and title
plt.xlabel("t")
plt.ylabel("x(t)")
plt.title(r"Solutions of $\dot{x}=k_1ax-k_{-1}x^2$")
plt.legend()
plt.grid()
plt.show()
```



Trajectories below $x^* = 1$ rise toward it and trajectories above it fall toward it, and $x_0 = 0$ stays at zero, as expected.

4.

I extend the setup from the text, but set $C = \frac{1}{5}$. Let v_C be the voltage across the capacitor, i_C be the current through the capacitor, v_R the voltage across the resistor, i_R the current through the resistor, v_L the voltage across the inductor, and i_L the current through inductor.

Let $x_1 = i_R = i_L = i_C$, let $x_2 = v_c$.

The system is

$$\begin{aligned} \frac{1}{5} \cdot \frac{dx_2}{dt} &= x_1 \\ v_r &= x_1^3 + 4x_1 \\ 1 \cdot \frac{dx_1}{dt} &= v_L \\ x_2 + v_R + v_L &= 0 \end{aligned}$$

which simplifies to

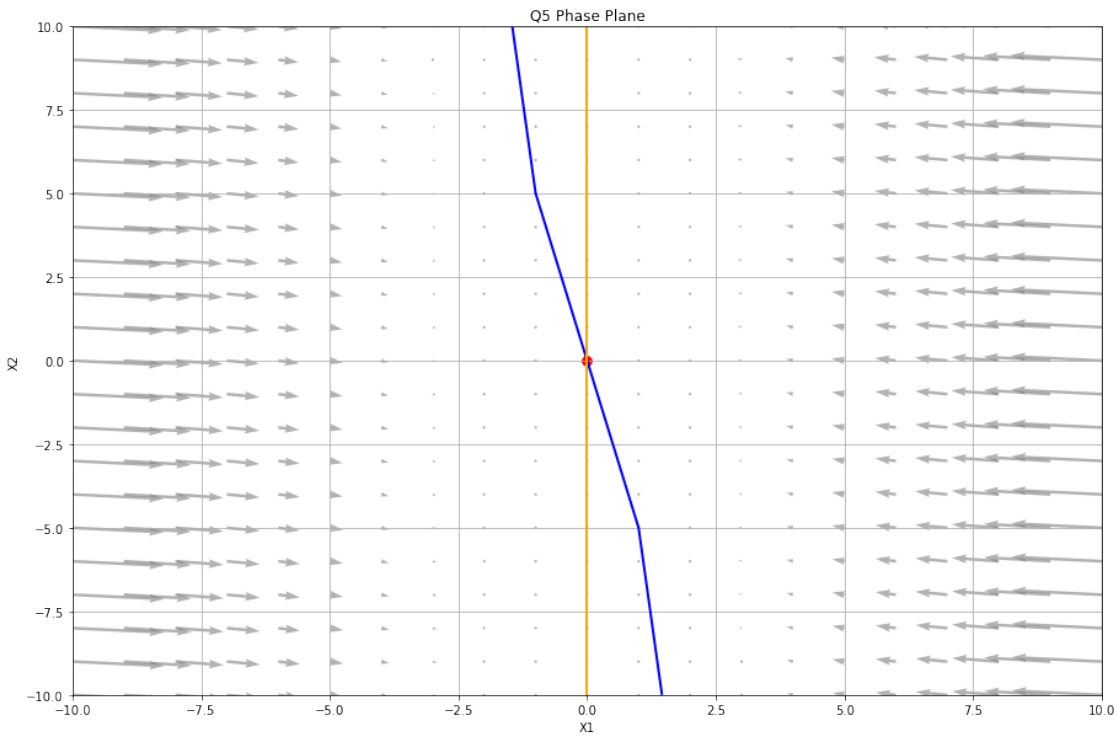
$$\begin{aligned} \frac{dx_1}{dt} &= -x_2 - x_1^3 - 4x_1 \\ \frac{dx_2}{dt} &= 5x_1. \end{aligned}$$

(a)

When $\dot{x_2} = 0$, $x_1 = 0$, which forces $x_2 = 0$. An equilibrium point is (0,0). The vector field for the model is below.

[8]:
```
# meshgrid for vector field
x1_vals = np.linspace(-10, 10, 21)
x2_vals = np.linspace(-10, 10, 21)
X1, X2 = np.meshgrid(x1_vals, x2_vals)

# evaluate diff eq on grid
dX1 = -X2 - X1**3 - 4*X1
dX2 = 5*X1

# plot vector field
plt.figure(figsize=(15,10))
plt.quiver(X1, X2, dX1, dX2, color="gray", alpha=0.6)

# plot nullclines where db=0, df=0
```

```
plt.contour(X1, X2, dX1, levels=[0], colors="blue", linewidths=2)
plt.contour(X1, X2, dX2, levels=[0], colors="orange", linewidths=2)

# scatter equilibrium points
plt.scatter(0, 0, marker="o", color="red", s=60)

# labels, title, grid
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Q5 Phase Plane")
plt.grid()
plt.show()
```



Flow rotates around the origin.

(b) I compute the Jacobian,

$$A = \begin{pmatrix} -3x_1^2 - 4 & -1 \\ 5 & 0 \end{pmatrix}.$$

At the equilbrium point (0,0),

$$A = \begin{pmatrix} -4 & -1 \\ 5 & 0 \end{pmatrix}.$$

I solve the characteristic equation $\mid A - \lambda I \mid = 0$

$$\begin{aligned}
\det \begin{pmatrix} -4-\lambda & -1 \\ 5 & -\lambda \end{pmatrix} &= 0 \\
(4+\lambda)(\lambda) + 5 &= 0 \\
\lambda^2 + 4\lambda + 5 &= 0 \\
\lambda_{1,2} &= \frac{-4 \pm \sqrt{16 - 4 \cdot 1 \cdot 5}}{2} \\
&= -2 \pm i.
\end{aligned}$$

The real component of the eigenvalues (-2) is negative, hence $\boxed{\text{the equilibrium point (0,0) is stable}}$. Because of the non-zero imaginary component, it is a stable spiral.

(c) I use $G(x) \approx A(x - x_0)$ to construct a linear system that approximates the behavior of the original nonlinear system,

$$\begin{aligned}
\frac{dx_1}{dt} &= -4x_1 - x_2 \\
\frac{dx_2}{dt} &= 5x_1.
\end{aligned}$$

I solve for the eigenvectors associated with eigenvalues $-2 \pm i$.

$$\begin{aligned}
\begin{pmatrix} -2-i & -1 \\ 5 & 2-i \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies x_1 = 1, \quad x_2 = -2 - i. \\
\begin{pmatrix} -2+i & -1 \\ 5 & 2+i \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies x_1 = 1, \quad x_2 = -2 + i.
\end{aligned}$$

Using the formula for the general solution, where

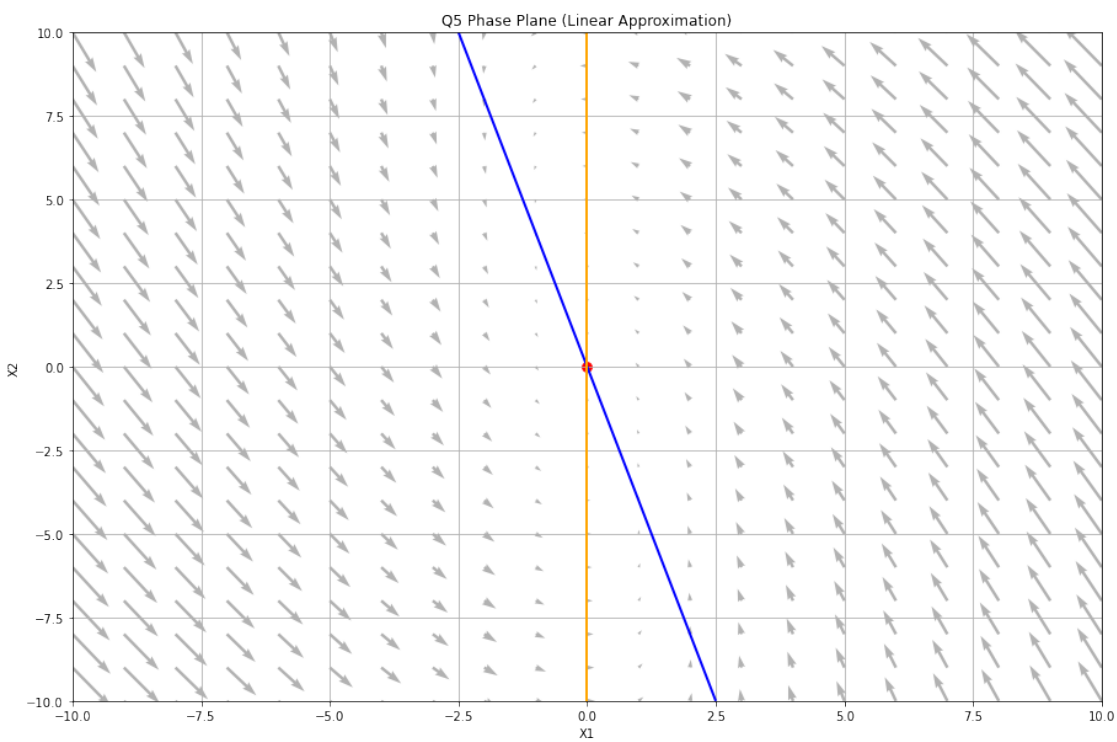
$$v_1 = \begin{pmatrix} 1 \\ -2-i \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \end{pmatrix} + i \begin{pmatrix} 0 \\ -1 \end{pmatrix},$$

I obtain

$$\boxed{x(t) = c_1 e^{-2t} \left[\begin{pmatrix} 1 \\ -2 \end{pmatrix} \cos(t) - \begin{pmatrix} 0 \\ -1 \end{pmatrix} \sin(t) \right] + c_2 e^{-2t} \left[\begin{pmatrix} 1 \\ -2 \end{pmatrix} \sin(t) + \begin{pmatrix} 0 \\ -1 \end{pmatrix} \cos(t) \right]}$$

[9]:
```
# evaluate diff eq on grid
dX1_approx = - 4*X1 - X2

# plot vector field
```

```
plt.figure(figsize=(15,10))
plt.quiver(X1, X2, dX1_approx, dX2, color="gray", alpha=0.6)

# plot nullclines where db=0, df=0
plt.contour(X1, X2, dX1_approx, levels=[0], colors="blue", linewidths=2)
plt.contour(X1, X2, dX2, levels=[0], colors="orange", linewidths=2)

# scatter equilibrium points
plt.scatter(0, 0, marker="o", color="red", s=60)

# labels, title, grid
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Q5 Phase Plane (Linear Approximation)")
plt.grid()
plt.show()
```



(d) See above for complete phase portraits. The nonlinear and linear portraits agree near the origin. The only equilibrium is (0,0). In the nonlinear portrait, the cubic term produces stronger damping for large values of x_1, so trajectories still go inwards.

When $C = 1/3$, the system's equilibrium is a stable node because the eigenvalues are negative with no imaginary component. Lowering the capacitance C still results in complex eigenvalues with a non-zero imaginary part and negative real part, thus the equilibirum point is a stable spiral.

Solution trajectories with lower C spiral into the equilibrium point and decay is oscillatory. The lower the capacitance C, the larger the imaginary component of the eigenvalues

$$\lambda = -2 \pm \sqrt{4 - \frac{1}{C}},$$

and thus the more pronounced the spiral.